

Microsoft Virtual Worlds Group

Microsoft Comic Chat Content Development

DRAFT 2: 7/18/26

1. Overview

Developing content for Comic Chat is a five-step process:

1. Create the designs for a background or character.
2. Scan the artwork, creating bitmap files.
3. Use the original bitmap images to create transparency and aura masks.
4. Create an .AVF file, which is a textual description of how the pieces of art come together to construct the character.
5. Compile the .AVF file and place it in the Comicart\Avatars directory.

The design process allows for two character formats, giving you control over how the personality of your character is expressed. Separate bitmap images enable your character to make standard gestures, as well as express a wide range of emotions. Once the data for a character is compiled, Comic Chat reads the compiled character information at run-time to determine how the character moves and expresses itself.

You can design a Comic Chat character to be either *unary* or *binary*. Bitmap images for a unary character portray the entire body, in various poses. Bitmap images for binary characters combine separate head poses and body gestures, allowing a broad range of expression with a moderate amount of original artwork.

What you'll need:

- a bitmap image for the background (optional)
- a 40x40 thumbnail sketch of your character
- foreground images of your character in several poses
- an aura mask for each foreground image
- a transparency mask for each foreground image
- a text file (.AVF file) that describes how the bitmap images for your character combine to form gestures

The sections which follow present guidelines for designing a Comic Chat character, and instructions for creating the .AVF text file.

1.1 Terms Used In This Document

unary character

A character whose original artwork portrays both head and body in a single bitmap for each gesture.

binary character

A character whose original artwork portrays head gestures in separate bitmap files from body gestures.

emotion wheel

The Comic Chat tool enabling the user to select a character's expression. The center of the emotion wheel represents a neutral posture. Positions on the outer edge of the wheel represent more extreme expressions of the selected emotion. Emotions and gestures are arranged around the outer edge of the circle.

icon

The term icon as used in this document refers to a 40x40 bit image that is used in the icon view of the Member List pane. It portrays a thumbnail sketch of the character, usually the head.

2. Designing a Character for Comic Chat

Artwork for a Microsoft Comic Chat environment consists of single-frame bitmap files: one for the background, and a collection of bitmap files for each character. How many bitmap files you need to create depends on the range of expressions you want the personality to express, and whether you choose to make the character unary or binary.

Bitmap files can be any size or shape, but when designing a character, keep in mind that up to five characters can appear in one frame. The system automatically scales artwork to fit inside a frame. Make sure the visual complexity of the background does not overpower the character design.

2.1 What files you need to create

For each Comic Chat character you create, you need at least the following files:

1. One bitmap file showing the character itself. If your character is binary, you need two bitmaps, one for the head and one for the body of your character.
2. A black-and-white transparency mask, to prevent the background from bleeding through your image. Binary characters need one transparency mask for each head and one for each body bitmap.
3. A black-and-white aura mask, to provide some white space around your character's figure. Binary characters need one aura mask for each head and one for each body bitmap.
4. a 40x40 thumbnail sketch of your character, for the Icon view of the Member List pane.
5. A textual description of your character and the files necessary to display it (.avf format).

All of these files are described in the sections which follow. With only one bitmap image, your character does not change from frame to frame during a chat session. To give life to your character, you need additional bitmaps that show your character expressing itself. Emotions and gestures are described in Section 2.3, "Define the personality's gestures and emotions."

2.2 Choose a unary or binary character

Unary characters are easier to create than binary characters. Jordan is an example of a unary character. Each expression and intensity is contained in one frame of one bitmap file. Each bitmap file shows the whole person, as shown in the following example:

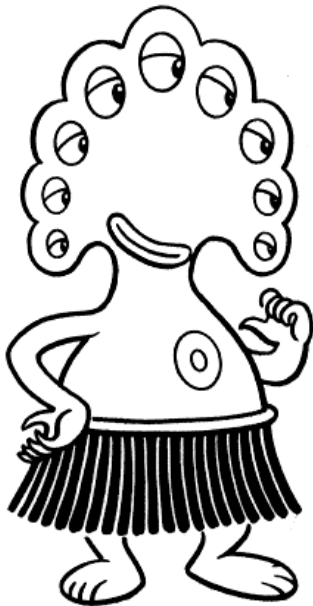


Figure 1: Jordan, a unary character

The binary character format can generate a large number of expressions from a minimum of artwork. Ten head designs and ten body designs can combine to make 100 possible gestures. You have control over how much random access is available to the program. Choosing a binary character also allows you to combine gestures such as pointing or waving with emotions such as laughter or sadness. The following example shows two bitmaps, one showing the head of a binary character, the second showing its torso:

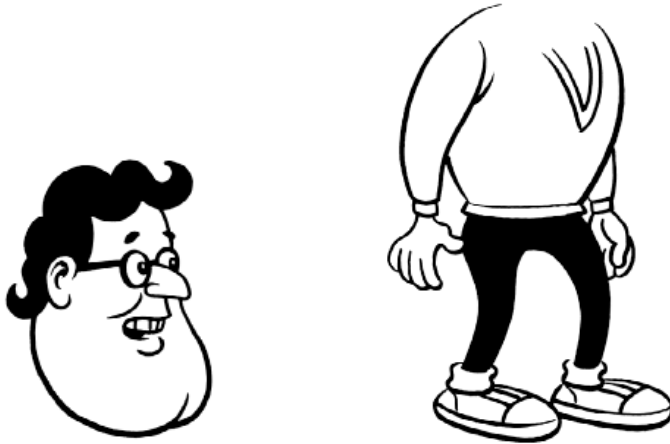


Figure 2: TongueTyed, a binary character

2.3 Define the personality's gestures and emotions

Before creating the artwork for a character, you need to define its personality. Keep in mind that end users select these images to represent themselves.

Your character expresses its personality through the following emotion keywords on the Comic Chat emotion wheel:

- Laugh
- Happy
- Coy
- Bored
- Scared
- Sad
- Angry
- Shout

In addition, you need to design the following gestures:

- at least three neutral poses (the system cycles randomly through these)
- a wave
- your character pointing to him or herself
- your character pointing to someone else

If you do not provide artwork for an emotion, Microsoft Comic Chat randomly selects from the available gestures. Create several neutral poses so the character does not remain static from frame to frame.

You also need a 40x40 bitmap file that contains a thumbnail sketch of your character. This image is used for the Icon view option for the Member List pane.

2.3.1 Planning Your Character's Poses

Microsoft Comic Chat automatically adjusts character position depending on how many characters are in one frame, and sometimes might flip an image from left to right. Because of this, the image needs to be bilaterally symmetric.

The system tries to make the characters look toward one another. For consistency, your characters should face either forward, or to their left (your right as you face the screen). A three-quarter turn generally appears more natural than a full profile.

Since most frames show only the head and shoulders of a character, it is best to avoid unusual or eccentric poses.

The system automatically scales images to fit the frame of the background. You can add white space above a character's head or below its feet in your original artwork, but extra white space is trimmed and discarded.

2.3.2 Special Considerations for Binary Characters

Plan an overlap design for binary characters. Sometimes a simple change in registration can produce a new expression. You describe the relative alignment of head and torso using specially calculated coordinates that are placed in the .avf file. For information on calculating x- and y-coordinates for binary characters, see the section "Construction of the AVF file."

The head and torso of binary characters have the same scale (DPI), but can be different resolutions.

In most cases, the system draws the torso first, then places the head. You can also design characters whose head needs to be placed before the torso (for example, if your character wears a high collar that covers the chin). Keywords in the .avf file specify whether head or torso needs to be drawn first. For information on what these keywords are, see the section "Header keywords and values."

2.3.3 Intensity Levels for Emotions and Gestures

When end users select an emotion from the emotion wheel, they choose an emotional intensity as well. You can create unresponsive characters by providing only a small range of intensity, or excitable characters by providing many levels of expression per emotion or gesture. Comic Chat reads intensity in tenths of a point on a scale from 0.0 to 1.0, where 0.0 is neutral and 1.0 is extreme. Because of the small size of the emotion wheel, it may be difficult for a user to select from more than four or five intensity levels. To keep your character interesting, provide from two to four images for each emotion or gesture.

Reserve the 0.0 intensity for neutral poses.

2.3.4 Using Color

On most systems, images appear in black-and-white even when the original bitmaps are in color. If a user's video card has a high color depth (more than 16 bit), it bypasses the lookup table and uses direct color.

2.4 Creating the Image Files

Use any program that creates bitmap (.bmp) files. The program scales the artwork to fit into the frames. Bitmap files can be Run-Length Encoded (RLE) for compression, or uncompressed (non-RLE).

Caution: Not all paint programs create the same RLE output. RLE files created by some programs cannot be read by Comic Chat. If you plan to use RLE files for compression, test your program's output to make sure it is recognized by Comic Chat. To avoid problems, use non-RLE files, or use Paint Shop Pro to convert a bitmap file to RLE format.

Selecting the size and format of your original bitmap is a tradeoff between resolution and download time. For best results, and to reduce download time and disk space for the end user, set the height from 220 to 440 pixels, with a width appropriate for your character.

Use a 1, 4, or 8-bit .bmp file for the foreground images, using 4-bit for gray scale. A 4-bit .bmp file, compressed, is more than twice the size of a 1-bit .bmp file. If your goal is a small compressed file (for example, 50k), you may need to use 1-bit black and white bitmaps. Conversion programs such as Debabelizer can change files from 4-bit to 1-bit format.

Transparency masks and aura masks need to be 1-bit black-and-white bitmap files. More information for creating transparency and aura masks is in the following section.

Use a standard naming convention to organize your bitmap files. The sample .AVF file at the end of this document uses the following naming convention:

- The first two characters are an abbreviation of the character's name.
- An underscore follows the abbreviation.
- A sequence number follows the underscore. Each sequence number maps to an emotion.
- An *m* or *a* follows the sequence number for transparency masks (m) and auras (a).

For example, one set of bitmap files for Jordan are named:

jn_1.bmp	foreground
jn_1a.bmp	aura
jn_1m.bmp	transparency mask

2.5 Transparencies and auras

For each character/gesture combination you create, you also need a bitmap file for the transparency mask and for the aura.

The *transparency mask* is a one-bit black and white bitmap file that is black in the areas where you want the character to be visible, and white where it should be transparent. Transparency masks prevent the background from showing through the character, and prevent head-body-aura interference in binary characters. The upper left pixel defines the transparency color.

The *aura* is a one-bit black and white mask file that creates a band of white space around the character's image. To construct an aura mask, begin with the transparency mask and add a black band around the outline, about four to ten pixels wide. Any white space becomes transparent in the final image.

TIP: If you are using Adobe Photoshop, you can use the Edit Stroke toolbar to create the aura.

You can use the aura file to achieve interesting effects, such as a wider aura around the character's head when it is angry, or a halo around the head of a very happy character.

The following images show a character, its transparency mask and aura mask. Note that the aura mask is identical to the transparency, and slightly larger.



Figure 3: Foreground image and transparency mask for unary character



Figure 4: aura mask for unary character (4 to 10 pixels wider than transparency mask)

3. Designing a Background for Comic Chat

You can substitute your own background instead of the one provided with Microsoft Comic Chat. The background is a simple bitmap file. You can create your own, or use any of the bitmaps provided by Microsoft Windows™ as a background. The system automatically scales the image to the appropriate size.

Place the background bitmap file in the Comicart\Backdrop folder of your Comic Chat directory. Select the background in Microsoft Comic Chat by clicking the Backgrounds tab of the View Options menu.

4. Getting the Images into Comic Chat

Once you have created the images for a character, the next step is to create an ASCII text file (.avf file) that describes how the pieces of art come together to construct your character. Use any text editor or word processor that creates text files. Give the .avf file the same name as the character; for example, the one for Jordan is named `Jordan.avf`. To see sample .avf files, see the end of this section.

The program `AvatarFiler.exe` compiles the .avf file into a format that Comic Chat can use. When you start this program, it opens a blank window. Use the File Open command to open your .avf file, then click File Save As to save it as a compiled .avb file. Give the file the same name as your character. For example, the files for Maynard are called `Maynard.avf` and `Maynard.avb`.

When `AvatarFiler` has created the .avb file, place it in the Comicart\Avatars folder of your Comic Chat directory. If you have created a new background bitmap, move the .bmp file for the background into the Comicart\Background folder of your Comic Chat directory.

4.1 Construction of the AVF file

You need to create one .avf file for each character you have created. Comments can appear anywhere in the .avf file. The pound sign (#) as the first character on the line indicates a comment. You can use either spaces or tabs interchangeably as data separators.

Each .avf file has the following structure:

1. The first line begins with the string "#AvFile" followed by two numbers:
 - 1 for binary characters, or 2 for unary characters
 - version number information (currently, the version is 1)

For example, the first line of the .avf file for Jordan would read:

```
#AvFile 2 1
```

2. Header information consisting of keywords and values, summarizing general information about the character and artist.
3. *For a unary character*, a section beginning with "BeginBodies" and ending with "EndBodies" (no quotation marks). This section describes the foreground file, transparency file, and aura file for each gesture for this character.

For a binary character, two sections: one delimited by "BeginFaces" and "EndFaces," and a second delimited by "BeginTorsos" and "EndTorsos" (no punctuation and no quotation marks). These sections describe the foreground, transparency mask, and aura mask for each head and torso for this character.

4. Each .avf file ends with the keyword "End."

The following sections provide details for each section of the .avf file. For a sample .avf file, see the end of this document.

4.1.1 Header keywords and values

The following keywords can appear in the header portion of a character's .avf file:

!ame

Enter the name of the character. Microsoft Comic Chat sessions show the character's name exactly as you type it, including capitalization.

Style

Enter a numeric value representing the background environment the character best fits in. At present, this value is 1.

Flags

For binary characters only. Enter one or more of the following keywords, separated by spaces, that describe how to draw the character:

TORSOFIRST indicates the torso is drawn first
 HEADMASK indicates a mask file is provided for the head
 HEADFIRST indicates the head is drawn first
 TORSOMASK indicates a transparency mask is provided for the torso

Desc

Enter a brief description of the character.

Artist

Enter the name of the artist.

Icon

Enter the name of the bitmap file that contains the icon for this character.

4.1.2 Emotion wheel keywords

Descriptive information is organized by emotion or gesture. This section immediately follows the header keywords. Information printed for each emotion and gesture is different, depending on whether your character is unary or binary. The keywords for emotions are gestures are:

EM_POINTOTHER
 EM_POINTSELF
 EM_NEUTRAL
 EM_WAVE
 EM_COY
 EM_ANGRY
 EM_BORED
 EM_LAUGH

EM_HAPPY
 EM_SCARED
 EM_SHOUT
 EM_SAD

The only required gesture is EM_NEUTRAL, with an intensity of 0.0. All other gestures and emotions are optional.

4.1.3 Describing unary characters in the .avf file

Each emotion or gesture has three lines of text: one for the foreground (six columns), one for the aura (two columns), and one for the transparency (two columns). Each line begins with one of the following keywords: FOREGROUND, AURA, or DITTO.

Column layout for unary character

Column 1 Keyword	Column 2	Column 3	Column 4	Column 5	Column 6
FOREGROUND	name of .bmp file	keyword for emotion or gesture	intensity value (in tenths)	center of head X-coordinate	center of head Y-coordinate
AURA	name of .bmp file	(blank)	(blank)	(blank)	(blank)
TRANSPARENCY	name of .bmp file	(blank)	(blank)	(blank)	(blank)
DITTO	(blank)	keyword for emotion or gesture	intensity value (in tenths)	(blank)	(blank)

The intensity value is a decimal value, ranging in tenths from 0.0 to 1.0. Use 0.0 only for the EM_NEUTRAL, EM_POINTOTHER, EM_POINTSELF, and EM_WAVE keywords.

For example:

```

FOREGROUND      jn_2.bmp      EM_POINTSELF 0.0      133      131
AURA           jn_2a.bmp
DITTO           EM_COY        0.8
  
```

You can use any number of spaces or tabs to separate the columns, but for readability, it is better to align the columns vertically, as shown in the examples.

You can use the keyword DITTO to use the same bitmap file for two gestures. In the previous example, the same image is used for EM_POINTSELF and EM_COY.

4.1.4 Describing binary characters in the .avf file

The .avf file for a binary character has two main sections following the header:

- 1) A section describing the character's head
 - begins with the keyword "BeginFaces"
 - three lines of text for each emotion/intensity level: one describing the foreground (ten columns), one naming the transparency bitmap (two columns), one naming the aura bitmap (two columns)
 - ends with the keyword "EndFaces"

- 2) A section describing the character's body
- begins with the keyword “BeginTorsos”
 - three lines of text for each emotion/intensity level: one describing the foreground (six columns), one naming the transparency bitmap (two columns), one naming the aura bitmap (two columns)
 - ends with the keyword “EndTorsos”

The ten columns describing the foreground for the head contains the following information:

Column layout for binary characters (head section)

Col. 1 keyword	Col. 2	Col. 3	Col. 4	Col. 5	Col. 6	Col 7	Col 8	Col 9	Col 10
FOREGROUND	name of .bmp file	emotion or gesture keyword	intensity value (in tenths)	DOT X	DOT Y	Delta X	Delta Y	Center of head X-coordinate	Center of head y-coordinate
AURA	name of .bmp file	(blank)	(blank)	(blank)	(blank)	(blank)	(blank)	(blank)	(blank)
TRANSPARENCY	name of .bmp file	(blank)	(blank)	(blank)	(blank)	(blank)	(blank)	(blank)	(blank)
DITTO	(blank)	emotion or gesture keyword	intensity value (in tenths)	(blank)	(blank)	(blank)	(blank)	(blank)	(blank)

The intensity is expressed as a decimal value, ranging in tenths from 0.0 to 1.0. Use 0.0 for the EM_NEUTRAL, EM_POINTOTHER, EM_POINTSELF, and EM_WAVE keywords.

DOT X and DOT Y are the x- and y-coordinates for the point where the head aligns with the body.

DELTA X and DELTA Y are calculated values. For an explanation of how to calculate these values, see “Calculating Head and Torso Coordinates for Binary Characters.”

Column layout for binary character (torso section)

Column 1 keyword	Column 2	Column 3	Column 4	Column 5	Column 6
FOREGROUND	Name of .bmp file	keyword for emotion or gesture	decimal value for intensity	TRUE X (calculated)	TRUE Y (calculated)
AURA	Name of .bmp file	(blank)	(blank)	(blank)	(blank)
TRANSPARENCY	Name of .bmp file	(blank)	(blank)	(blank)	(blank)
DITTO	(blank)	keyword for emotion or gesture	decimal value for intensity	(blank)	(blank)

The intensity is expressed as a decimal value, ranging in tenths from 0.0 to 1.0. Use 0.0 for the EM_NEUTRAL, EM_POINTOTHER, EM_POINTSELF, and EM_WAVE keywords.

For an explanation of how to calculate values for head and torso coordinates, see the following section, “Calculating Head and Torso Coordinates for Binary Characters.”

You can use any number of spaces or tabs to separate the columns, but for readability, it is better to align the columns vertically, as shown in the examples.

You can use the keyword DITTO to use the same bitmap file for two gestures.

A sample .avf file for TongueTied, a binary character, is shown at the end of this document.

4.1.5 Calculating Head and Torso Coordinates for Binary Characters

This section describes how to normalize the head and body bitmaps against each other and arrive at the relative coordinates recorded in the .avf file.

For each head and body image, select a point where the head aligns with the torso. The X and Y coordinates for this point are labeled NEW X and NEW Y.

Align each head and torso image into a single image. Find the actual points where the head and torso line up with one another. The X and Y coordinates for this point are called DOT X and DOT Y. Use the DOT X and DOT Y values of the first head as the BASE X and BASE Y values for the torso. Use the DOT X and DOT Y values of the first torso as the BASE X and BASE Y values for the head.

Use either a spreadsheet or a calculator to calculate the following values:

$\text{DELTA X}(\text{unit}) = \text{DOT X}(\text{unit}) \text{ minus BASE X}$

$\text{DELTA Y}(\text{unit}) = \text{DOT Y}(\text{unit}) \text{ minus BASE Y}$

In this formula, *unit* refers to either the head or torso unit. Each head and each torso has a delta x and delta y value calculated for it. Base X and Base Y remain constant for all torsos and heads. See the sample spreadsheet for an example.

Find the center of the head and record the X and Y coordinates in a value called FACE X and FACE Y.

True X and True Y for the torso are calculated according to the following formula:

$\text{True X} = \text{NEW X} \text{ minus DELTA X} \text{ minus (FACE X for the corresponding head)}$

Example

The following example shows part of the spreadsheet for TongueTied, a binary character.

Head Frame	Base X Base Y		(Body1 = Neutral)					
	Dot X	Dot Y	Delta X	Delta Y			Face X	Face Y
Head1	240	337	12	14	104	171	87	109
Head2	235	328	7	5	96	171	90	118
Head3	228	332	0	9	107	170	96	119
Head4	238	335	10	12	110	164	90	116
Head5	220	312	-8	-11	107	158	110	112
Head6	230	329	2	6	112	167	90	109
Head7	229	343	1	20	96	175	82	114
Head8	224	331	-4	8	97	168	88	116
Head9	232	339	4	16	84	175	68	115
Head10	232	334	4	11	83	170	67	113
Head11	227	333	-1	10	104	173	87	111

Body Frame	Base X Base Y		(Head 1)					
	Dot X	Dot Y	Delta X	Delta Y	New X	New Y	True X	True Y
Body1	228	323	-12	-14	123	20	123	20
Body2	224	325	-16	-12	87	27	91	25

Body3	219	332	-21	-5	128	64	137	55
Body4	218	336	-22	-1	134	41	144	28
Body5	223	335	-17	-2	117	61	122	49
Body6	222	341	-18	4	198	38	204	20
Body7	229	334	-11	-3	103	34	102	23
Body8	239	327	-1	-10	160	43	149	39
Body9	220	322	-20	-15	133	29	141	30
Body10	219	318	-21	-19	110	33	119	38
Body11	229	327	-11	-10	147	38	146	34

Compare these figures with the values in the sample .avf file for TongueTyed, shown in the following section.

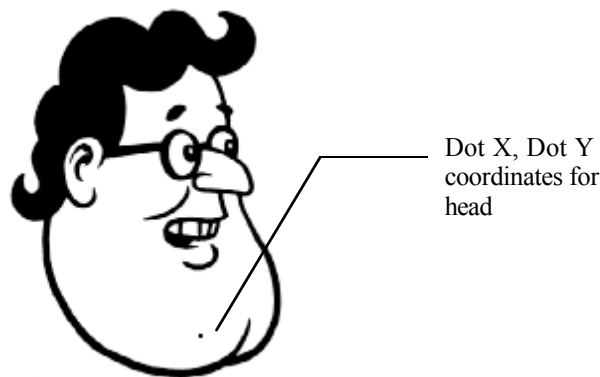


Figure 5: Dot X, Dot Y coordinates for head of binary character

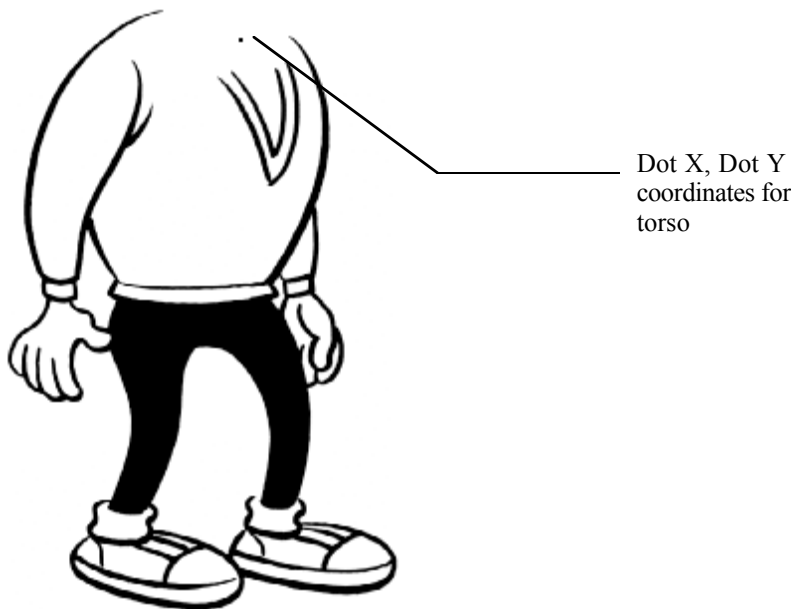


Figure 6: Dot X and Dot Y coordinates for torso of binary character

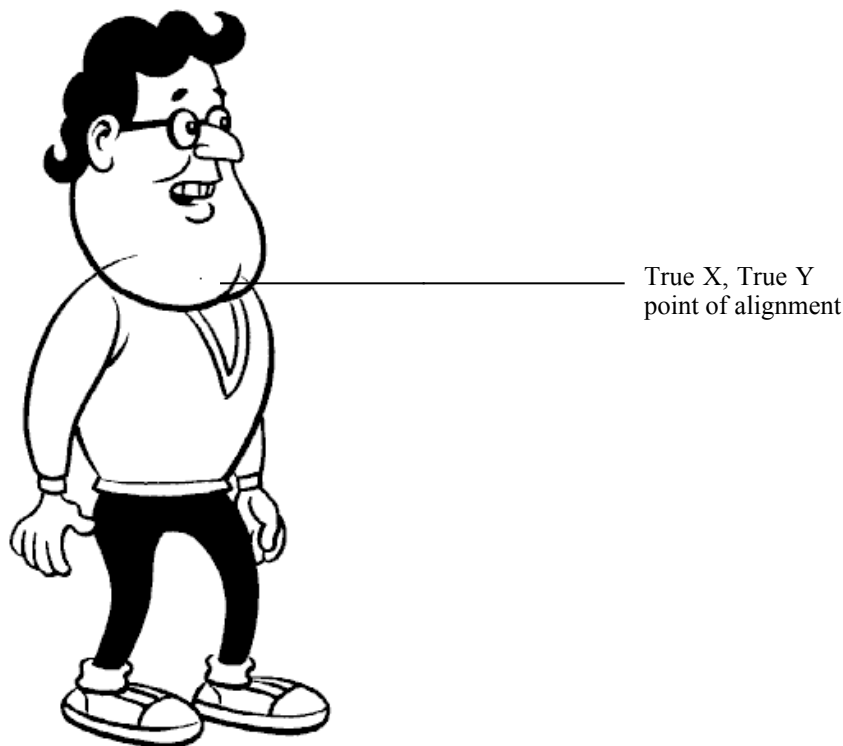


Figure 7: Head and torso of aligned binary character

4.1.6 Sample .avf files

The following example shows the .avf file for Jordan, a unary character.

```
#AvFile 2      1
Name           Jordan
Style          1
Flags          TORSOFIRST HEADMASK
Desc           This is Jordan
Artist         Jim Woodring
Icon           jn_icon.bmp

BeginBodies

FOREGROUND     jn_1.bmp      EM_POINTOTHER 0.0    133    131
AURA          jn_1a.bmp

FOREGROUND     jn_2.bmp      EM_POINTSELF 0.0     133    131
AURA          jn_2a.bmp

DITTO          EM_COY 0.8

FOREGROUND     jn_3.bmp      EM_ANGRY      1.0     133    131
AURA          jn_3a.bmp

FOREGROUND     jn_4.bmp      EM_BORED      0.8     133    131
AURA          jn_4a.bmp
```

FOREGROUND	jn_5.bmp	EM_LAUGH	1.0	133	131
AURA	jn_5a.bmp				
FOREGROUND	jn_6.bmp	EM_WAVE	0.0	133	131
AURA	jn_6a.bmp				
FOREGROUND	jn_7.bmp	EM_NEUTRAL	0.0	133	131
AURA	jn_7a.bmp				
DITTO		EM_HAPPY	0.4		
FOREGROUND	jn_9.bmp	EM_NEUTRAL	0.0	133	131
AURA	jn_9a.bmp				
# Not very scared, but better than random neutrals					
DITTO		EM_SCARED	0.2		
FOREGROUND	jn_11.bmp	EM_NEUTRAL	0.0	133	131
AURA	jn_11a.bmp				
FOREGROUND	jn_12.bmp	EM_SHOUT	1.0	133	131
AURA	jn_12a.bmp				
DITTO		EM_SAD	1.0		
EndBodies					
END					

The following example shows an .avf file for a binary character.

```
#AvFile 1      1
Name           TongueTyed
Style          1
Flags          TORSOFIRST HEADMASK
Desc           This is TongueTyed
Artist         Jim Woodring

BeginFaces

FOREGROUND     tt_h1.bmp      EM_NEUTRAL      0.0      104      171      12      14      87
109
TRANSPARENCY   tt_h1m.bmp
AURA          tt_h1a.bmp

FOREGROUND     tt_h2.bmp      EM_HAPPY        0.2      96      171      7      5      90
118
TRANSPARENCY   tt_h2m.bmp
AURA          tt_h2a.bmp

# Also Neutral
DITTO          EM_NEUTRAL      0.0

# Also Coy
DITTO          EM_COY        0.5
```

FOREGROUND	tt_h3.bmp 119	EM_LAUGH	0.8	107	170	0	9	96
TRANSPARENCY	tt_h3m.bmp							
AURA		tt_h3a.bmp						
FOREGROUND	tt_h4.bmp 116	EM_ANGRY	0.4	110	164	10	12	90
TRANSPARENCY	tt_h4m.bmp							
AURA		tt_h4a.bmp						
# Above can also be neutral?								
#FOREGROUND	tt_h5.bmp 112	DOUBLE	0.0	107	158	-8	-11	110
#TRANSPARENCY	tt_h5m.bmp							
#AURA		tt_h5a.bmp						
FOREGROUND	tt_h6.bmp 109	EM_SAD	0.8	112	167	2	6	90
TRANSPARENCY	tt_h6m.bmp							
AURA		tt_h6a.bmp						
FOREGROUND	tt_h7.bmp 114	EM_ANGRY	0.8	96	175	1	20	82
TRANSPARENCY	tt_h7m.bmp							
AURA		tt_h7a.bmp						
# Also a poor excuse for shout...								
DITTO		EM_SHOUT	0.8					
FOREGROUND	tt_h8.bmp 116	EM_ANGRY	0.6	97	168	-4	8	88
TRANSPARENCY	tt_h8m.bmp							
AURA		tt_h8a.bmp						
# Front shot								
FOREGROUND	tt_h9.bmp 115	EM_NEUTRAL	0.0	84	175	4	16	68
TRANSPARENCY	tt_h9m.bmp							
AURA		tt_h9a.bmp						
# Front shot								
FOREGROUND	tt_h10.bmp 113	EM_HAPPY	0.7	83	170	4	11	67
TRANSPARENCY	tt_h10m.bmp							
AURA		tt_h10a.bmp						
FOREGROUND	tt_h11.bmp 111	EM_HAPPY	0.8	104	173	-1	10	87
TRANSPARENCY	tt_h11m.bmp							
AURA		tt_h11a.bmp						
EndFaces								
BeginTorsos								
FOREGROUND	tt_b1.bmp	EM_NEUTRAL	0.0	123	20			
#TRANSPARENCY	tt_b1m.bmp							

AURA	tt_b1a.bmp				
# Also minor angry					
DITTO	EM_ANGRY	0.4			
BACKGROUND	tt_b2.bmp	EM_NEUTRAL	0.0	91	25
#TRANSPARENCY	tt_b2m.bmp				
AURA	tt_b2a.bmp				
# Also minor angry					
DITTO	EM_ANGRY	0.2			
BACKGROUND	tt_b3.bmp	EM_WAVE	0.0	137	55
#TRANSPARENCY	tt_b3m.bmp				
AURA	tt_b3a.bmp				
BACKGROUND	tt_b4.bmp	EM_POINTSELF	0.0	144	28
#TRANSPARENCY	tt_b4m.bmp				
AURA	tt_b4a.bmp				
#BACKGROUND	tt_b5.bmp	EM_DOUBLEPOINT	0.0	122	49
#TRANSPARENCY	tt_b5m.bmp				
#AURA	tt_b5a.bmp				
BACKGROUND	tt_b6.bmp	EM_SHRUG	0.0	204	20
#TRANSPARENCY	tt_b6m.bmp				
AURA	tt_b6a.bmp				
BACKGROUND	tt_b7.bmp	EM_POINTOTHER	0.0	102	23
#TRANSPARENCY	tt_b7m.bmp				
AURA	tt_b7a.bmp				
BACKGROUND	tt_b8.bmp	EM_ANGRY	0.8	149	39
#TRANSPARENCY	tt_b8m.bmp				
AURA	tt_b8a.bmp				
BACKGROUND	tt_b9.bmp	EM_NEUTRAL	0.0	141	30
#TRANSPARENCY	tt_b9m.bmp				
AURA	tt_b9a.bmp				
# Also moderate anger					
DITTO	EM_ANGRY	0.6			
BACKGROUND	tt_b10.bmp	EM_NEUTRAL	0.0	119	38
#TRANSPARENCY	tt_b10m.bmp				
AURA	tt_b10a.bmp				
# Also moderate coy					
DITTO	EM_COY	0.5			
EndTorsos					
Icon	tt_icon.bmp				
END					